

Jennifer (LLM of Choice: Claude)

“Can you create an example website for my project? this is what my project is. could you generate several pages of the ui for each of the pages?”

Gitto is a web-based social media platform designed to help users document, track, and share personal projects and goals from inception to completion. The system will allow users to: Create and manage personal accounts (Login Page) Post and document projects Break projects into milestones Track progress over time View a personalized content feed (Feed Page) Add comments and reactions to posts View and manage user profiles (User Profile Page) Search for other users (User Search Function) The overall goal of Gitto is to foster a community of motivated individuals who can share progress, receive feedback, and maintain accountability while pursuing personal or professional projects.”

- Me nor my teammates are particularly savvy with UI/UX design so I used this prompt to generate a styled, modern, jsx layout with static mock data. It allowed us to first decide on If we liked how everything looked before integrating the frontend. I was able to adjust the coloring, sizing on the title and just general preference fixes.

“This is meant to be a group project so several people will be working on different pages. What is a good file structure for me to separate this content into feed page / milestone profile / search / login / auth?”

- Based on what we agreed on splitting for each page on the project, I asked it to generate the file structure that we would be working with just to better implement the single responsibility principle on the code.

“I would like to proceed with the frontend / backend structure for this project. I want to start the project from scratch and create a new repo on github for the team to use. we planned on using react, next, tailwind css for the frontend and node / express for the backend. For the storing we would use mongodb. later on we will use google auth / github auth for sign ins.”

- I prompted the LLM with a complete overview of our plans for this project including the technologies I would use. This prompt allowed me to build up the project repo, initialize the read me, install all project dependencies and connect it to github and etc. I then wrote out the file structure it provided prior and omitted anything I knew we would not use.
This prompt also converted the frontend it gave me above from the fixed/ static inline styling into tailwind css so I was able to copy that code. The complete frontend styling was built out with this single prompt and some styling issues I was having before with the pages not adjusting to different screen sizes and etc. were all fixed once converting to tailwind.
- This was helpful for my team to be able to focus heavily on the backend development and simply attach it to buttons that were already created on the front.

“How do I know if im connected to the correct mongosh server?”

- The first thing I did was start server.js in the backend and connect the project to the mongodb database. I was able to go online and add the atlas cluster and populate my env with the secret

key credentials but I was unsure how to connect to mongosh in the terminal to the specific server. This gave me the correct instructions and I was able to verify the connection status to the correct database server and that our database was up and running as expected

“give me steps to adding google auth to this project”

- After implementing out JWT sign in, We knew we wanted 2 more options for the user to sign in using github and their google accounts. This is my first time adding OAuth credentials to a project so this gave me directions to create the project in google cloud, pull the google sign in API and generate my secret key for usage. It then helped generate the backend/lib/passport.js page needed to find and create the user in their respective databases.

“okay i am ready to implement github oath sign in. I am trying to learn the oath path so correct me if im wrong anywhere, Flrst, I need to login to github and obtain a github id and a github secret. Then I save both of those and add them into my. env files for my front and backend. Then I need to import the github provider into my frontend/src/app/api/auth/nextauth/route.js file and attach a new callback for github. I believe I can use the previously written callback file in backend/src/controllers/auth.controller.js on the github as well as google.

as for the passport file, i see the passport function is being called when using the get router in backend/src/routes/auth.routes.js

should i separate my passport files for the different providers or what do you recommend?”

- This was written in a way for me to get a better understanding of adding the OAuth API's to my website. I was already able to implement most of these except the passport file. The passport file was written in a way where it only satisfied the google route and was not open for extension / easy modifications for multiple OAuth Api's. I rewrote the passport file and created a helper function that was listed “findorCreateUser” that would take in the following parameters:

(provider, providerId, name, email, avatar)

And then wrote each Google and Github passport configuration, each calling on the findorcreateUser and inserting their own variables as needed.

“The only thing left to do is the search page and that is dependent on both the user page and the project page. Lets proceed with making the user page but we will come back to connecting the milestones and such later.”

- I wanted the LLM to help me generate a page on the user profile page that added the count for the user followers/ following and the cards for the user's personal projects. It already showcased the profile photo and username for the user on the page so it was more so just adding details.

“I am trying to pull my teammates work they did on the project-page branch and i am getting this error:

PS C:\Users\jenni\Gitto\Gitto> git pull project-page
fatal: 'project-page' does not appear to be a git repository fatal: Could not read from remote repository. Please make sure you have the correct access rights and the repository exists. PS C:\Users\jenni\Gitto\Gitto>

- It was time to merge the project page and milestone page made by my teammate and I was trying to debug issues I was experiencing when trying to pull my teammates branch to my local code editor. This was able to provide instructions to create a new local branch and force pull the changes that were pushed remotely so it could be viewed on my local editor.

“if i wanted to integrate ai into my gitto project, what could an ai chatbot be useful for within the app? with the pages i have available now the feed, profile, search and etc. unless i can add a new feature to it where ai could thrive?”

- I was thinking of ways to elevate the application in any way. So I asked the bot to generate some ideas in which AI could work really well on the app. I will likely add project overview / summarization for each project and milestone, so when people are quickly scrolling through the feed they can just take a look at the summarization and keep moving if they are interested / uninterested.

Axel Verdugo (LLM of Choice: Cursor)

For this project I leveraged Cursor’s planning function, so, although some prompts may seem simple, they were further broken down and structured until proper implementation was reached.

“Check the user/auth flow for any instance where fetching the users could cause the users to not exist at times.”

- Led to identifying and patching missing backend user endpoints (/api/users/search, /api/users/:handle), which directly affected user discoverability/profile fetch behavior.

“Implement the Post Comment Surface Map plan.”

- Resulted in post comment functionality being wired end-to-end (including read path + UI usage).

“I want the comment count button to send the user to the project/id updates tab...”

- Led to deep-link behavior from feed comment count to project page updates context.

“Implement the Feed Comment Deep Link plan.”

- Finalized routing with query params and focused/targeted update behavior.

“I want to implement milestone reaching... authenticated user can mark a Milestone as complete.”

- Introduced owner-only milestone completion flow (MVC path).

“Implement the Milestone Completion MVC Plan.”

- Applied backend route/controller + frontend controls for marking milestones complete.

“Currently, progress bar does not move according to milestone completion...”

- Prompted root-cause analysis that identified missing project progress recalculation coupling.

“Implement the Progress Recalculation Policy Options plan.”

- Added automatic project progress recalculation on milestone mutations/completions.

“Currently, when there are no posts in the Updates Tab... add a button... takes you to feed and autoselects this project.”

- Drove empty-state CTA + feed projectId preselection behavior.

“Implement the Empty Updates Feed CTA Plan.”

- Finalized owner-only CTA in empty updates state and feed query-param preselect fallback behavior.

“Run tests on all of these features, ensuring consistent functionality, no linter problems, and correct MCV structure.”

- Triggered full test/lint/MVC validation workflow and fixes to make checks pass.

“Fetch all of the currently existing references, api calls, routes, or components that reference the ‘edit’ page.”

- Triggered a full trace/mapping of edit-related frontend/backend wiring and gaps.

“Let’s work specifically on the edit project page... edit button should be isOwner limited... open into a new page... create a TEMPLATE... edit project name, milestone names, project description.”

- Defined the actual feature scope and behavior for the new edit project page.

“Implement the plan as specified...”

- Triggered implementation of the owner-gated edit route/template and redirect notice flow.

“Next step, wiring... explain in detail exactly what is wired and where, API calls and such.”

- Drove the detailed architecture/API wiring breakdown before save functionality.

“Implement now.”

- Triggered phase-2 save wiring (PATCH /api/projects/:id) and success-state UX wiring.

Bug report prompt:

“Cannot access ‘loadUpdateComments’ before initialization... occurs after editing and then clicking on updates.”

- Triggered runtime fix for hook initialization order in UpdatesTab. Audit current project/createproject state.

“Check status of createproject and project pages: what’s implemented vs incomplete, and current structure.”

- Plan and implement project page backend connectivity

“Create a plan for backend connectivity and UI-only buttons on project page, including star/unstar.”

- Included your scope choices: follow/unfollow, share, add milestone, updates like/comment. Create a test project first.

“Before project-page work, create a test project for manual visual/backend testing.”

- Execute the project connectivity plan.

“Implement the attached plan fully; mark todos in progress/completed.”

- This drove adding missing project/post endpoints and wiring project-page actions. Run frontend/backend for testing.

“Run both front and back end so I can test; ensure access to the test project.”

- Fix live runtime regressions.
Reported issues and prompted fixes:
Next dynamic params Promise issue (params.id/params.username needs use(params)).
Updates tab crash from rendering milestone object as React child.

“Project not found” + 400/404 diagnosis.

- Plan and implement project-page error handling

“Create an error handling checklist.”

Then: “Implement checklist fully with todos.”

- This changed load/action/tab error states and retry/session UX.

“Plan and implement project/feed parity (exclude profile)”

“Create building plan for project/feed parity; profile is out of scope.”

Then: “Implement attached parity plan fully with todos.”

- This drove missing project/post lifecycle APIs and parity validation.

“Plan and implement Saved feature (MVC)”

“Saved seems easy; check current code first.”

Then: “Proceed” to planning.

Then: “Implement saved feature plan fully with todos.”

- This added persistent save/unsave backend routes, serializer state, service wiring, bookmark behavior, and /saved page. Validation and safe remote push workflow

“Run validation tests (API + lint) and safely push to remote project-page branch.”

Then: “Implement attached validation/push plan fully with todos.”

“Using my currecnt account, create a project I can use to make a post. Nothing specific, just a dummy project since I dont have the project creation page set up yet.”

“Using the plan we discussed, implement a valid create project routing, project controller, and middleware to normalize and validate the project creation as outlined above. Ensure you add concise comments on every major addition to ease the work flow. Then, perform a simple test to ensure it works as defined above.”

"Perfect, can those rules be moved into backend/src/middleware/validate.middleware? Explain changes"

"Let's plan it out, however, don't forget to explain reasoning and changes proposed. Lets keep in mind, auth is not my part, so give my teammates code priority over mine unless change would only improve performance"

"Make a plan for implementing the function for these buttons, ensuring to follow the MCV structure we spoke of before and keeping non-feed sections as clean and unchanged as possible"